CHAPTER 2
SYSTEM ANALYSIS

2.1 Existing System and Proposed System

At present, candidates prepare for interviews through printed question banks, websites, video tutorials, coaching classes and mock interviews with friends or mentors. A few mobile applications and online platforms also offer practice questions and automated mock interviews. Almost all of these depend on spoken communication: questions are asked aloud and answers are expected to be spoken. This works for many candidates, but it leaves out people who are Deaf, Hard of Hearing or Speech Impaired. The main limitations of the existing system are:

1. Sign language, text and audio are not offered together, so users cannot choose the way in which they want to answer.
2. Mentors who understand sign language are rare, and coaching is difficult to arrange and costly.
3. Feedback is delayed or missing, and scores are not given in a clear and consistent form.
4. Questions are not arranged by round and topic, and the progress of a candidate is not recorded.

InterviewSign is proposed to remove these limitations. The only thing a user needs is a mobile phone. After logging in, the user selects an interview round, an option and one or more topics. Each question is shown on the screen and is also read aloud, so that it can be followed by reading or by listening. The user can then answer by typing text, by recording a sign language video or by giving an audio answer. After the answer is submitted, the application gives feedback with a score, and the attempt is saved so that it appears in the history and in the statistics on the home screen.

The proposed system also provides registration and login with an email address or a username, a live check of username availability, a profile with a photograph, and a controlled username change policy. Its advantages are that it is accessible to hearing- and speech-impaired users, available at any time without the need for a human mentor, gives immediate and consistent feedback, shows improvement through scores and streaks, and keeps data secure because every user can access only his or her own information.

2.2 Requirement Analysis

Requirement analysis identifies what the system must do and the conditions under which it must work. The functional requirements of InterviewSign are:

1. The system shall allow a new user to register with name, unique username, email address and password, and optional details such as phone number, city, hearing or speech status and answer preference.
2. The system shall allow login with either an email address or a username and shall show a generic error message for invalid credentials.
3. The system shall let the user select an interview round, an option and one or more topics before starting an interview.
4. The system shall display each question on the screen and read it aloud.
5. The system shall accept the answer as typed text, a recorded sign language video or an audio recording.
6. The system shall give feedback and a score for each answer and store the attempt for the user.
7. The system shall show attempt history with category filters and statistics such as attempts, average score and streak.
8. The system shall let the user view and edit the profile, upload a profile photograph and log out.

The non-functional requirements are:

1. Accessibility: the interface uses clear icons, high-contrast colours and large touch areas, and offers text, video and audio ways of answering.
2. Security: passwords are managed by Supabase Authentication, screens are protected by login, and users can change only their own data.
3. Performance: screens open quickly and username checks are delayed by 500 milliseconds so that unnecessary requests are avoided.
4. Usability and portability: the application is simple to navigate and runs on both Android and iOS from a single codebase.

To use the application, a person needs only a smartphone running Android or iOS with a screen, speaker, microphone and camera, together with an internet connection. No other device is required. The application is developed using React Native with Expo and

TypeScript, with expo-router for navigation and React Context for state. Supabase provides authentication, a PostgreSQL database and storage, and the code is written in Visual Studio Code.

React Native with Expo was selected because a single TypeScript codebase produces both Android and iOS applications, and Expo makes testing on real phones easy. TypeScript reduces errors through static typing, and expo-router makes protected navigation simple. Supabase offers ready-made authentication, a database with row level security and file storage, which removes the need to build and host a separate server and keeps the project secure and scalable.

The application needs permission to use the camera for sign video, the microphone for audio answers and the photo library for the profile photograph. The data to be stored consists of the profile details of each user, the attempts with category, date, answer type and score, and the profile photographs, all of which are kept in Supabase.

The feasibility of the system was also studied. Technically, all the required tools are free to start with and well documented. Economically, no special device or server has to be purchased, because the free plans of Expo and Supabase are enough for development and demonstration. Operationally, the application is easy for users to adopt, since it runs on the phone they already own and its screens follow common mobile patterns.